Cryptography

Public-Key Cryptography – RSA

Cristhian Iza

School of Mathematical and Computational Sciences
Yachay Tech University

September 20, 2026

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Public-Key Cryptography

Public-Key Cryptography



UC 2.

Public-Key Cryptography and Key Establishment

- Introduction to Public-Key Cryptography
- Number Theory and Modular Inverses
- RSA Cryptosystem
- RSA Implementation and Security
- Diffie–Hellman Key Exchange
- Key Establishment and Security Attacks

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Bruce Schneier



Twofish,
Blowfish

Vincent Rijmen
and Joan Daemen



AES

Modern private
key encryption

Rivest, Shamir
& Aldeman



Public-k
encrypt

Ron
Rivest



Hashing

Whitfield
Diffie



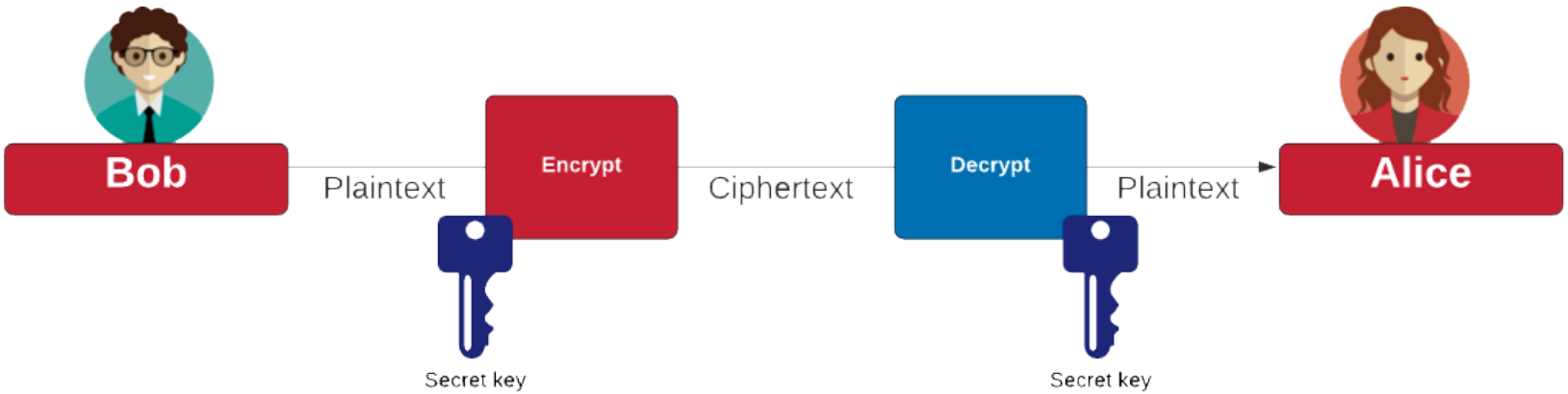
Key
interchange

Phil
Zimmerman



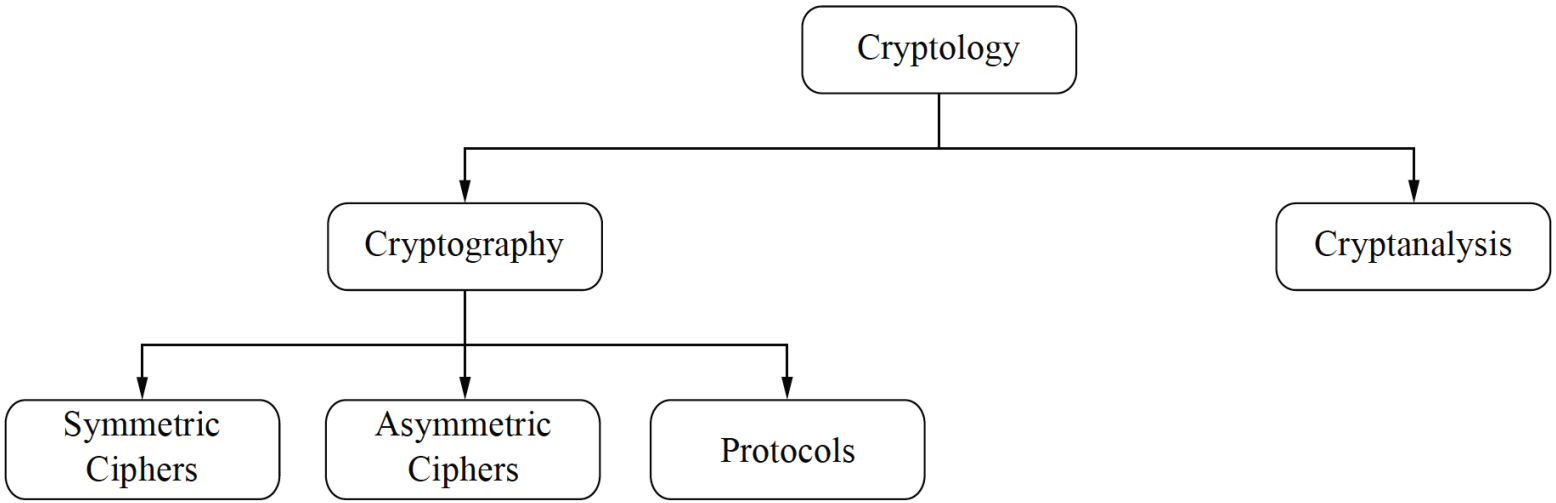
PGP
Encryption

Limitations of Symmetric Cryptography



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Introduction to Public-Key Cryptography

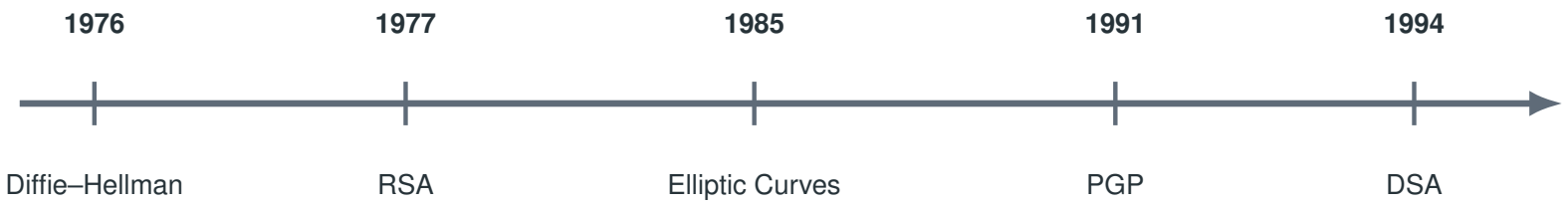


SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Evolution of Public-Key Cryptography

1976: New concept
Public-key cryptography and secure key establishment

Digital signatures and public-key systems become practical security tools



Main families:
RSA $\longrightarrow$ integer factorization
ECC $\longrightarrow$ elliptic-curve discrete logarithm

Historical Impact

Public-key cryptography enabled **key establishment, encryption, and digital signatures** without requiring a previously shared secret key.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Public-Key Algorithm Families

Integer Factorization

Hard problem

Factor $n = pq$ for large primes p and q.

Example
RSA

Uses
Encryption, signatures

Discrete Logarithm

Hard problem

Given $g^x \equiv y \pmod{p}$, find x.

Examples
Diffie–Hellman, ElGamal, DSA

Uses
Key establishment, encryption, signatures

Elliptic Curves

Hard problem

Given $Q = kP$, find k.

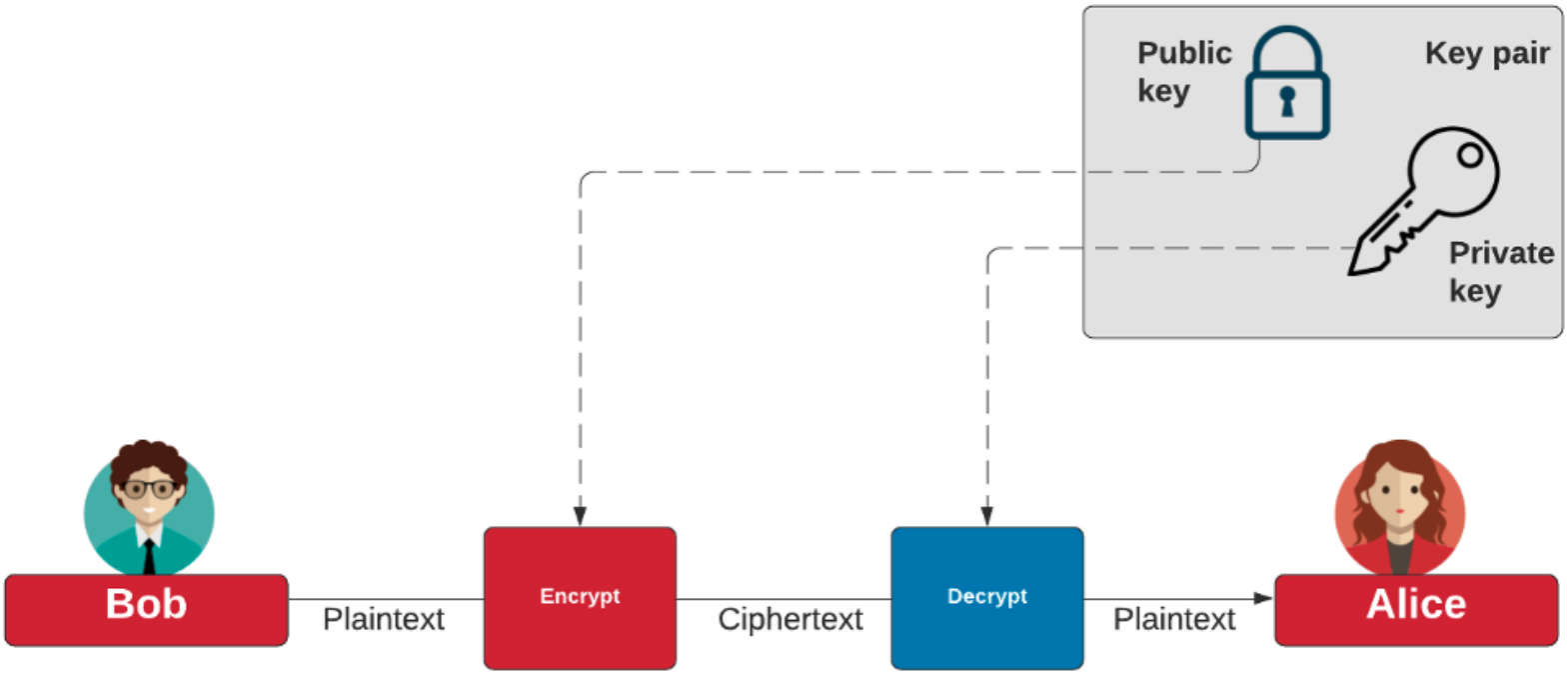
Examples
ECDH, ECDSA

Uses
Key establishment, signatures

Each family relies on a computationally hard mathematical problem.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Asymmetric Encryption: 1. Key Pair

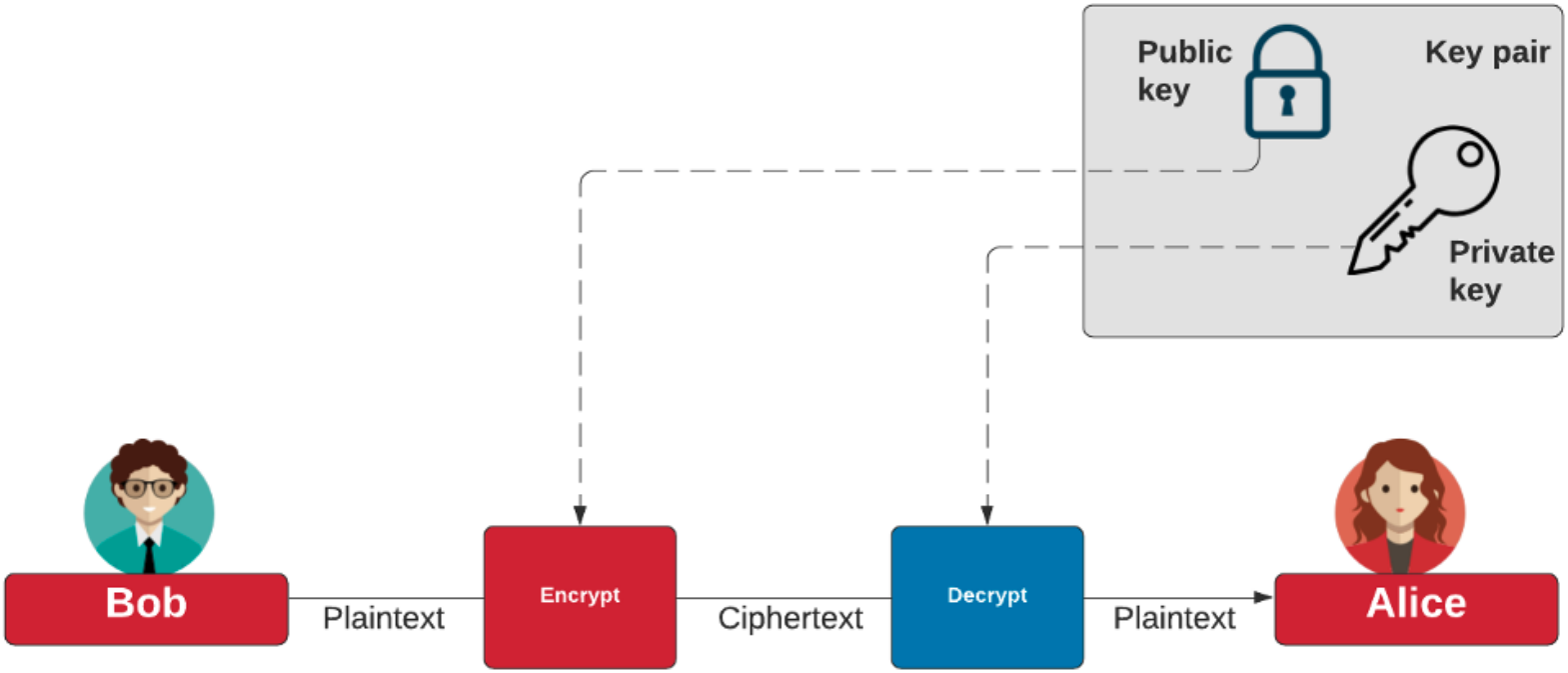


Key Pair Generation

- Alice generates two mathematically related keys.
- The **public key** may be shared.
- The **private key** must remain secret.

AND COMPUTATIONAL SCIENCES

Asymmetric Encryption: 2. Encryption



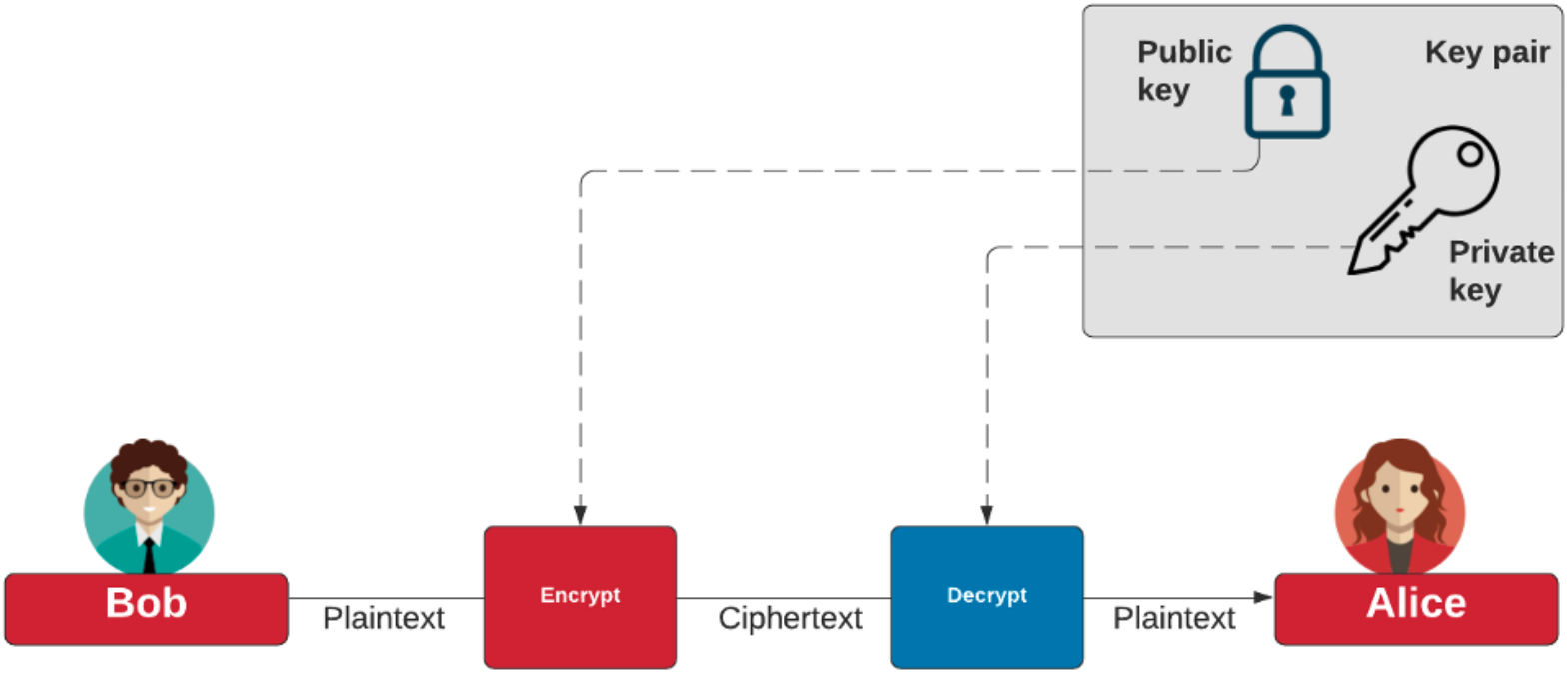
Bob Encrypts for Alice

- Bob obtains Alice's public key $K_{PU,A}$.
- Bob encrypts the plaintext message M.
- The output is ciphertext C.

$$C = E(K_{PU,A}, M)$$

AND
L SCIENCES

Asymmetric Encryption: 3. Decryption



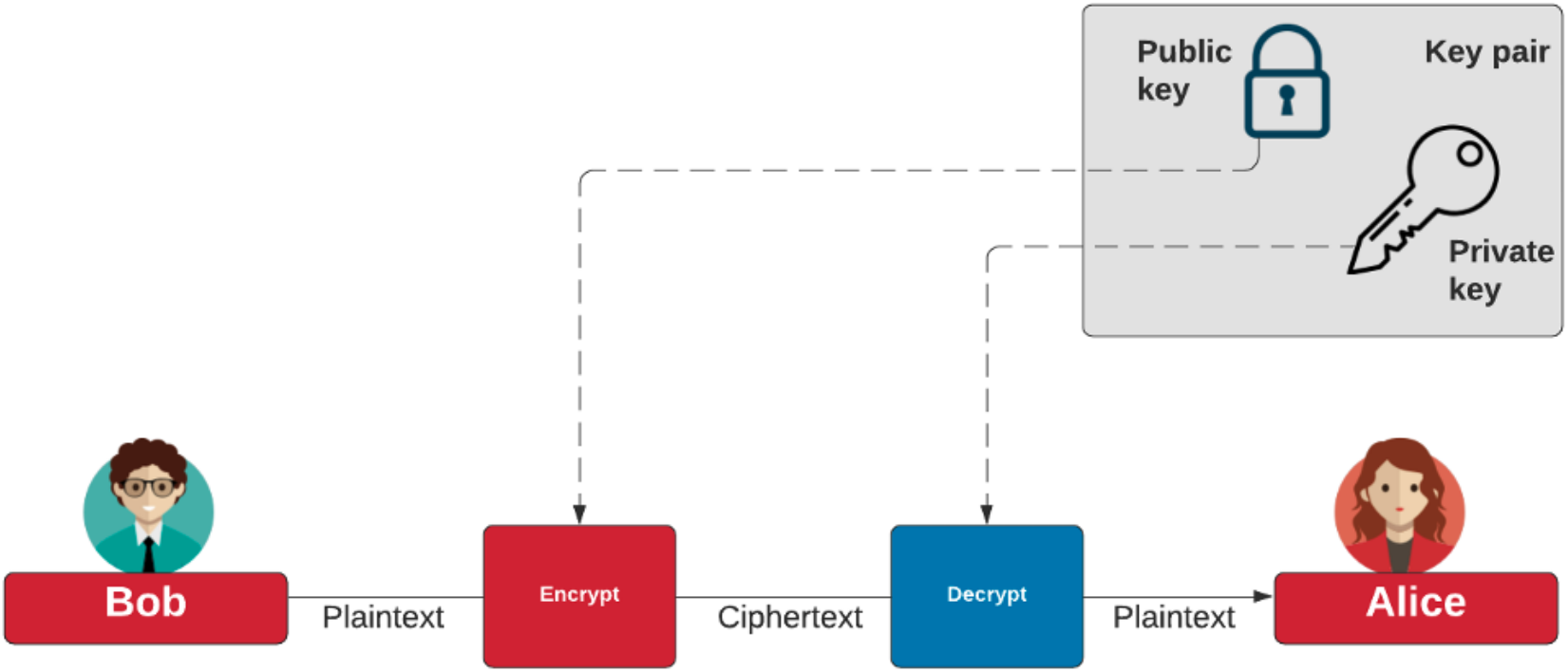
Alice Decrypts

- Alice receives ciphertext C.
- Alice uses her private key $K_{PR,A}$.
- The original plaintext message M is recovered.

$$\boxed{M = D(K_{PR,A}, C)}$$

AND
L SCIENCES

Public-Key Encryption

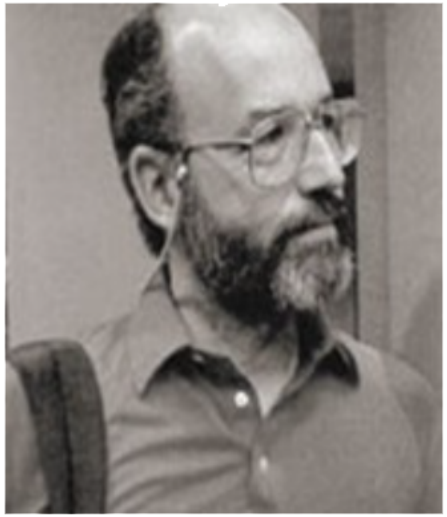


SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Rivest–Shamir–Adleman (RSA)



Ron Rivest



Adi Shamir



Leonard Adelman

In 1977, **Ron Rivest, Adi Shamir, and Leonard Adleman** introduced RSA, one of the most influential public-key cryptosystems.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA: Mathematical Foundation



- Multiplying two large primes is easy; factoring their large product is computationally difficult.
- **Euler's theorem** and **Euler's phi function** provide the number-theoretic basis used in the introductory RSA derivation.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA: Message Space

RSA works with integers modulo n. A plaintext is represented by an integer m such that

$$0 \leq m < n.$$

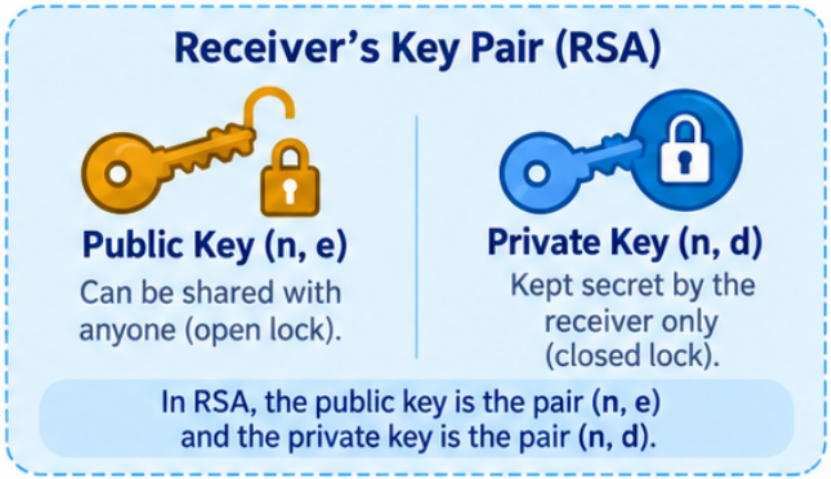
The ciphertext is also an element of $\mathbb{Z}_n$.

Practical rule

Do not encrypt arbitrary files directly with RSA. Real systems encode a short value with secure padding and normally use RSA only to protect a symmetric key or another small secret.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Encryption



Given the public key

$$k_{pub} = (n, e),$$

and a message representative $m \in \mathbb{Z}_n$, compute

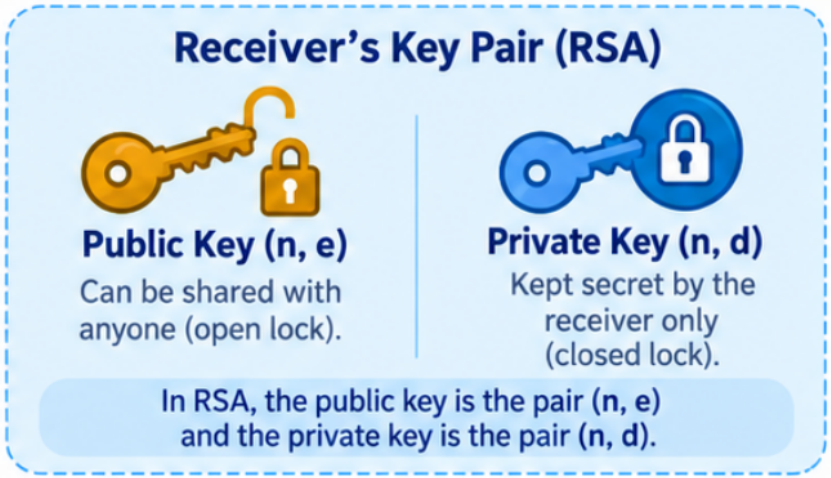
$$\boxed{c \equiv m^e \pmod{n}}.$$

Input: plaintext representative m

Output: ciphertext c

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Decryption



Given the private key

$$k_{pr} = (n, d),$$

and the ciphertext $c \in \mathbb{Z}_n$, compute

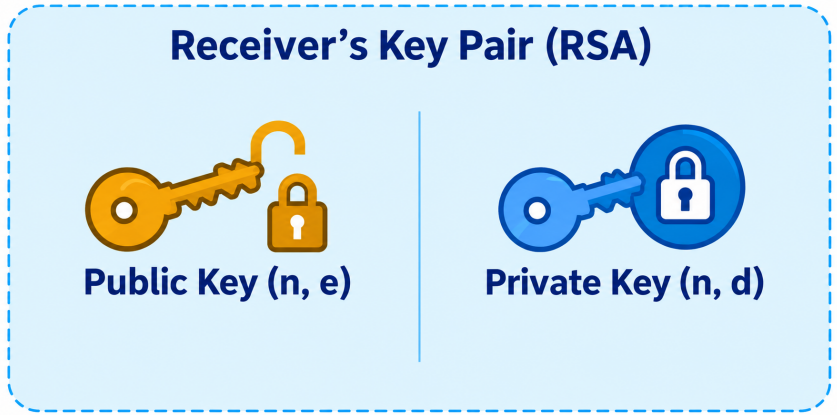
$$\boxed{m \equiv c^d \pmod{n}}.$$

Input: ciphertext c

Output: recovered message representative m

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Key Structure



Shared Parameter

Both keys contain the modulus n. The exponent e is public; the exponent d must remain secret.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Security Requirement

Core Security Requirement

Given only the public key (n, e), recovering the private exponent d must be **computationally infeasible**.

Why?

If an attacker factors $n = pq$, then the attacker can compute $\phi(n)$ and derive d.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Message-Size Constraint

Textbook RSA Constraint

The encoded message is an integer m in the range

$$0 \leq m < n.$$

Therefore, the RSA operation is defined only for values smaller than the modulus.

Practical RSA

Secure padding such as OAEP adds encoding overhead, so the real payload is **smaller than the modulus size**. RSA is not designed to encrypt large files directly.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA in Hybrid Encryption

1. Generate a random symmetric key.
2. Encrypt the data with a symmetric cipher.
3. Use RSA only to protect the short symmetric key.

RSA protects the key; the symmetric cipher protects the data.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA and Fast Modular Exponentiation

The Computational Task

RSA repeatedly evaluates

$$c \equiv m^e \pmod{n}, \qquad m \equiv c^d \pmod{n}.$$

Computing the full powers first is impractical for real key sizes.

Solution

Use **fast modular exponentiation**, which reduces intermediate results modulo n at every step. The Square-and-Multiply algorithm is the basic method studied next.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Encryption Formula

RSA Public Key

$$(n, e)$$

Plaintext

$$m$$

$$\Downarrow$$

$$c = m^e \bmod n$$

$$\Downarrow$$

Ciphertext

$$c$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Decryption Formula

RSA Private Key

$$(n, d)$$

Ciphertext

$$c$$

$$\Downarrow$$

$$m = c^d \bmod n$$

$$\Downarrow$$

Plaintext

$$m$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Key Generation (cont.)

1. Choose two large prime numbers:

$$p \qquad \text{and} \qquad q$$

2. Compute the modulus:

$$n = p \cdot q$$

3. Compute Euler's totient:

$$\phi(n) = (p-1)(q-1)$$

4. Choose the public exponent $e \in \{2, \ldots, \phi(n)-1\}$, such that:

$$1 < e < \phi(n), \qquad \gcd(e, \phi(n)) = 1$$

5. Compute the private exponent d:

$$d \equiv e^{-1} \pmod{\phi(n)}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Key Generation (cont.)

Public Key	Private Key
$k_{pub} = (n, e)$	$k_{pr} = (n, d)$

$$p, q \longrightarrow n \longrightarrow \phi(n) \longrightarrow e \longrightarrow d$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Exercises

RSA Exercise

Alice wants to use RSA to send a secure message.

Use the following values:

$$p = 11, \qquad q = 17, \qquad e = 7$$

The plaintext is:

$$m = 42$$

Tasks:

1. Compute n and $\phi(n)$.
2. Find the public key (n, e).
3. Find the private exponent d and the private key (n, d).
4. Encrypt the plaintext $m = 42$.
5. Decrypt the ciphertext and verify the original message.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Step 1

We start with:

$$p = 11 \qquad \text{and} \qquad q = 17$$

Compute the modulus:

$$n = p \cdot q$$

$$n = 11 \cdot 17$$

$$\boxed{n = 187}$$

Compute Euler's totient:

$$\phi(n) = (p-1)(q-1) = 160$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Step 2

The selected public exponent is:

$$e = 7$$

We must verify:

$$1 < e < \phi(n)$$

Therefore:

$$1 < 7 < 160$$

We also verify that e and $\phi(n)$ are coprime:

$$\gcd(7, 160) = 1$$

Therefore, $e = 7$ is valid.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Step 3

We need to find d such that:

$$de \equiv 1 \pmod{\phi(n)}$$

Using our values:

$$7d \equiv 1 \pmod{160}$$

Therefore:

$$d \equiv 7^{-1} \pmod{160}$$

Use the Extended Euclidean Algorithm:

$$\boxed{d = 23}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Step 4

Build the Private Key

$$n = 187 \qquad \text{and} \qquad d = 23$$

Therefore: $k_{pr} = (187, 23)$

Public Key	Private Key
$(187, 7)$	$(187, 23)$

Check:

$$7 \cdot 23 = 161$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Step 5

The plaintext is:

$$m = 42$$

The public key is:

$$(n, e) = (187, 7)$$

RSA encryption uses:

$$c = m^e \bmod n$$

Therefore:

$$c = 42^7 \bmod 187$$

Compute the powers:

$$\boxed{c = 15}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Step 6

The ciphertext is:

$$c = 15$$

The private key is:

$$(n, d) = (187, 23)$$

RSA decryption uses:

$$m = c^d \bmod n$$

Therefore:

$$m = 15^{23} \bmod 187$$

$$\boxed{m = 42}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise

Alice wants to create a small RSA system.
Use the following prime numbers:

$$p = 3, \qquad q = 11$$

The plaintext is:

$$m = 4$$

Consider the following possible values for the public exponent:

$$e \in \{2, 3, 4, 5\}$$

Tasks:

1. Compute n and $\phi(n)$.
2. Select a valid value for e.
3. Compute the private exponent d.
4. Write the public and private keys.
5. Encrypt the message $m = 4$.
6. Decrypt the ciphertext.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Solution (cont.)

1. Compute n and $\phi(n)$

$$n = pq = (3)(11) = \boxed{33}$$

$$\phi(n) = (p-1)(q-1) = (2)(10) = \boxed{20}$$

2. Select e

$$\gcd(e, 20) = 1$$

From $\{2, 3, 4, 5\}$, the valid value is:

$$\boxed{e = 3}$$

3. Compute d

$$ed \equiv 1 \pmod{20}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Solution (cont.)

$$3d \equiv 1 \pmod{20}$$

Since:

$$3(7) = 21 \equiv 1 \pmod{20}$$

then:

$$\boxed{d = 7}$$

4. RSA Keys

$$\boxed{k_{pub} = (33, 3)} \qquad \boxed{k_{pr} = (33, 7)}$$

5. Encryption

$$c = m^e \bmod n$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Solution (cont.)

$$c = 4^3 \bmod 33 = 64 \bmod 33$$

$$\boxed{c = 31}$$

6. Decryption

$$m = c^d \bmod n$$

$$m = 31^7 \bmod 33$$

Since:

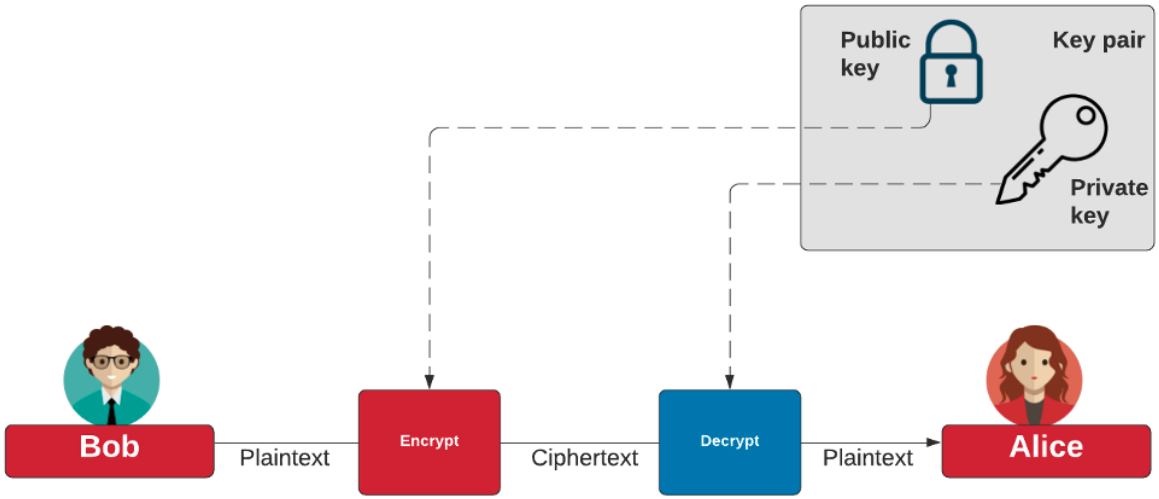
$$31 \equiv -2 \pmod{33}$$

$$31^7 \equiv (-2)^7 = -128 \equiv 4 \pmod{33}$$

Therefore: $m = 4$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise



RSA Encryption

- Bob receives the following RSA public key: $k_{pub} = (3233, 17)$. He also intercepts the ciphertext: $c = 855$ During a security incident, the following value is leaked: $\phi(n) = 3120$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise (cont.)

Tasks:

1. Identify the values of n and e.
2. Use the Extended Euclidean Algorithm to compute the private exponent d.
3. Write the private key k_{pr}.
4. Decrypt the ciphertext $c = 855$.
5. Verify the result by encrypting the recovered message again.

Question:
Why is knowing $\phi(n)$ a serious security problem in RSA?

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Solution (cont.)

1. Identify the public values
From:

$$k_{pub} = (3233, 17)$$

we obtain:

$$\boxed{n = 3233} \qquad \boxed{e = 17}$$

2. Compute the private exponent
We need:

$$17d \equiv 1 \pmod{3120}$$

Using the Extended Euclidean Algorithm:

$$\boxed{d = 2753}$$

The private key is:

$$\boxed{k_{pr} = (3233, 2753)}$$

3. Decrypt the ciphertext

$$m = c^d \bmod n$$

Substitute the values:

$$m = 855^{2753} \bmod 3233$$

Using modular exponentiation:

$$\boxed{m = 123}$$

4. Verify the result
Encrypt $m = 123$ again:

$$c = m^e \bmod n$$

$$c = 123^{17} \bmod 3233$$

$$\boxed{c = 855}$$

The ciphertext is the same.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Solution (cont.)

Security idea

If an attacker knows $\phi(n)$, the attacker can compute:

$$d = e^{-1} \pmod{\phi(n)}$$

and recover the private key.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Overview

RSA

openssl genrsa -out private.pem 1024

$C = M^e \bmod N$

$D = C^d \bmod N$

$N = p\,q$

cat private.pem

```
-----BEGIN RSA PRIVATE KEY-----
MIICXQIBAAKBgQC3qXK4kCxn3BNk87vJUMwIznU8pTjr10Kma9+Jkj4zEy/fiZtY
xvdmn1rKNq/8fEUDCcRVC8hQBpevqxFiJ3dbA7ZM6VjUAmztOfRfxSezgvkjswVS
F1/cgBM32AB4nx1dkCV/Wgedn3KFIFU+b8LH1ZLoyRMyLnwWmAkT/mBC/QIDAQAB
AoGAE8Yao+Rh44y+SdA0F6irTwdrd+wSBNJYSrKyjo1ARR97uAWIxDYnzNS7Yaoh
qH14sKsMiFuMZZFQI4m3hWnaX7OFjhJvxKjP6+BdXKsnwWxpwec7RS6n9ptA7qlF
aIFfVARyiWjG+q+8Bg8CTaHjGgtYPnfLzJM0Vef6gKg5vgECQQDZSKGxtdbpXwXw
VAC78SyfOOYmWKL1HiZs0nyTOnZmhMSkE4+S38zhDTjITh0cuKTksTFeUku/sRij
4T4Y9iz5AkEA2GMpeeRT3IQntmzQgTc7Rgez73Y/UWFynuErg++9gzI758TO3AoV
lFs4NOUAqhZ5fdwizs6sa0bjYm+BC1mbJQJBAMQVts4QItVSSqK6vDrfh/xctd4v
KUh5oAWe4otfPBCCio7jlDLgwxzp+K9TRxRvUWeMvNe4/uEMKgdiss6GAskCQQCf
MpVZMDriifgNppDgABqDszcWfhCnduI1McQqFT+APn0ETy9Bg8nMlDAN+k061b4c
ctDJBhSj+EtiKFbwWsRhAkAnEPn+6m3djTwJMw82DxK1q2fcIjTR0ng8pyrF2iIR
P7oBP8I4hGix/FOrV8M8virK6iCsslEcZBo39FkEqc0N
-----END RSA PRIVATE KEY-----
```

openssl rsa -in private.pem -text

```
Private-Key: (1024 bit)
modulus:
    00:b7:a9:72:b8:90:2c:67:dc:13:64:f3:bb:c9:50:
    cc:08:ce:75:3c:a5:38:eb:d7:42:a6:6b:df:89:92:
    3e:33:13:2f:df:89:9b:58:c6:f7:66:9f:5a:ca:36:
    af:fc:7c:45:03:09:c4:55:0b:c8:50:06:97:af:ab:
    11:62:27:77:5b:03:b6:4c:e9:58:d4:02:6c:ed:39:
    f4:5f:c5:27:b3:82:f9:23:b3:05:52:17:5f:dc:80:
    13:37:d8:00:78:9f:1d:5d:90:25:7f:5a:07:9d:9f:
    72:85:20:55:3e:6f:c2:c7:d5:92:e8:c9:13:32:2e:
    7c:16:98:09:13:fe:60:42:fd
publicExponent: 65537 (0x10001)
privateExponent:
    13:c6:1a:a3:e4:61:e3:8c:be:49:d0:34:17:a8:ab:
    4f:07:6b:77:ec:12:04:d2:58:4a:b2:b2:8e:8d:40:
    45:1f:7b:b8:05:88:c4:36:27:cc:d4:bb:61:aa:21:
    a8:7d:78:b0:ab:0c:88:5b:8c:65:91:50:23:89:b7:
    85:69:da:5f:b3:85:8e:12:6f:c4:a8:cf:eb:e0:5d:
    5c:ab:27:c1:6c:69:c1:e7:3b:45:2e:a7:f6:9b:40:
    ee:a9:44:68:81:5f:54:04:72:89:68:c6:fa:af:bc:
    06:0f:02:4d:a1:e3:1a:0b:58:3e:77:cb:cc:93:34:
    55:e7:fa:80:a8:39:be:01
prime1:
    00:d9:48:a1:b1:b5:d6:e9:5f:05:f0:54:00:bb:f1:
    2c:9f:38:e6:26:58:a2:f5:1e:26:6c:d2:7c:93:3a:
    76:66:84:c4:a4:13:8f:92:df:cc:e1:0d:38:c8:4e:
    1d:1c:b8:a4:e4:b1:31:5e:52:4b:bf:b1:18:a3:e1:
    3e:18:f6:2c:f9
prime2:
    00:d8:63:29:79:e4:53:dc:84:27:b6:6c:d0:81:37:
    3b:46:07:b3:ef:76:3f:51:61:72:9e:e1:2b:83:ef:
    bd:83:32:3b:e7:c4:ce:dc:0a:15:94:5b:38:34:e5:
    00:aa:16:79:7d:dc:22:ce:ce:ac:6b:46:e3:62:6f:
    81:0b:59:9b:25
```

L AND COMPUTATIONAL SCIENCES

RSA with OpenSSL: Key Generation

1. Generate a 2048-bit Private Key

```
openssl genpkey -algorithm RSA -pkeyopt
rsa_keygen_bits:2048 -out private_Bob.pem
```

2. Inspect the Private Key

```
openssl pkey -in private_Bob.pem -text -noout
```

3. Export the Public Key

```
openssl pkey -in private_Bob.pem -pubout -out
public_Bob.pem
```

Public key: share it. **Private key: keep it secret.**

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA with OpenSSL: Encryption with OAEP

Encrypt with Bob's Public Key

```
openssl pkeyutl -encrypt -pubin -inkey
public_Bob.pem
-in message.txt -out message.rsa
-pkeyopt rsa_padding_mode:oaep
```

What Happens?

- OpenSSL encrypts the input using **Bob's public key**.
- OAEP adds randomized padding before the RSA operation.
- The output is stored in `message.rsa`.

Do not use textbook RSA directly for real applications.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA with OpenSSL: Decryption with OAEP

Decrypt with Bob's Private Key

```
openssl pkeyutl -decrypt -inkey private_Bob.pem
-in message.rsa -out recovered.txt
-pkeyopt rsa_padding_mode:oaep
```

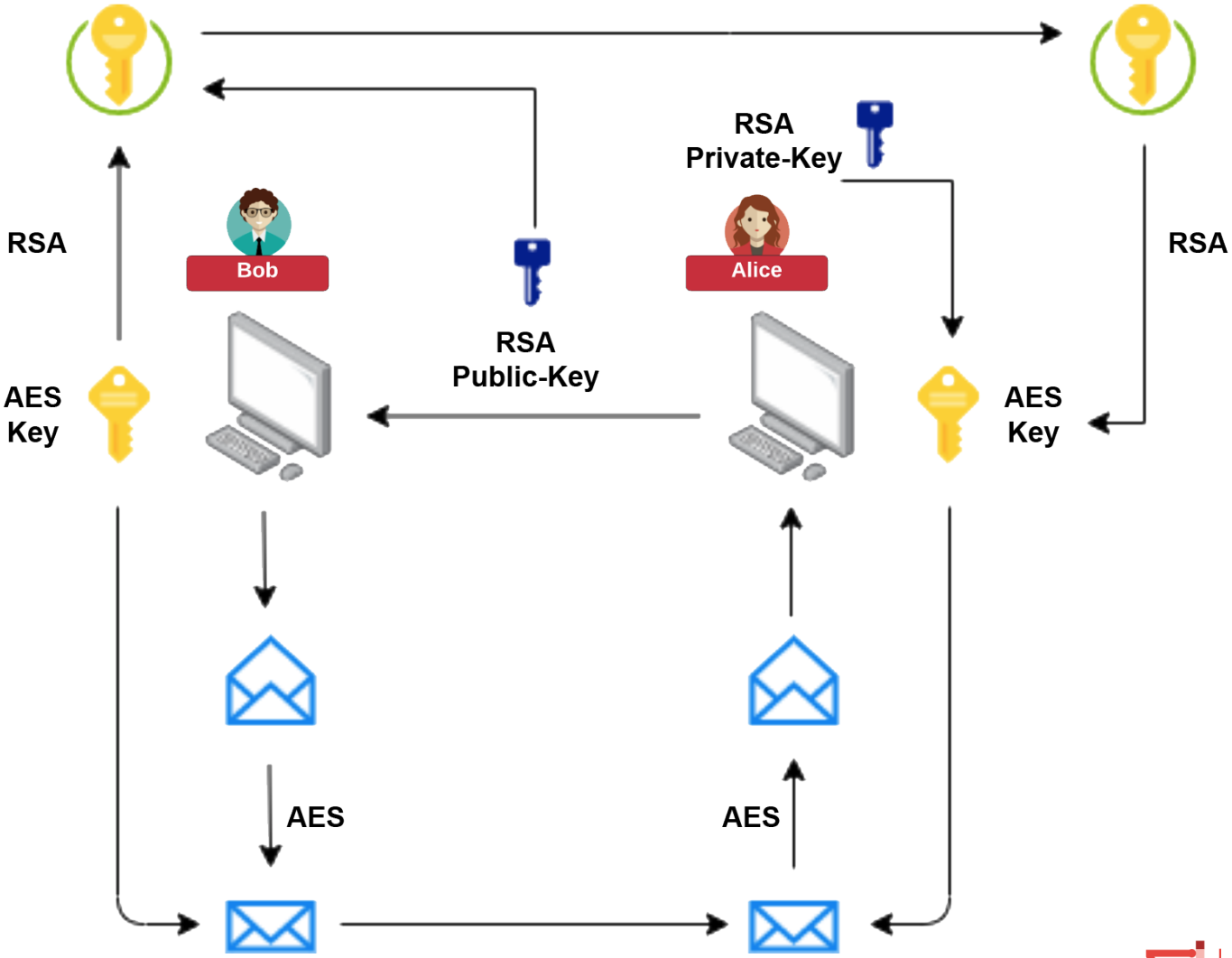
What Happens?

- OpenSSL uses **Bob's private key** to recover the message.
- The same padding mode must be used for encryption and decryption.
- The recovered plaintext is stored in `recovered.txt`.

In practice, RSA is commonly used to protect keys rather than large files.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Fast Modular Exponentiation

RSA Fast Modular Exponentiation

Why It Matters

Real RSA uses a large modulus n. A common public exponent is

$$e = 65537.$$

The private exponent d is much larger and must remain secret.

Efficient Methods

- Square-and-Multiply
- Montgomery exponentiation
- Chinese Remainder Theorem-CRT
- Hardware acceleration

RSA Operations

$$c \equiv m^e \pmod{n}, \qquad m \equiv c^d \pmod{n}.$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Square-and-Multiply Algorithm (cont.)

- Consider the exponentiation:

$$x^{26}$$

- Convert the exponent to binary:

$$26 = (11010)_2$$

- Define the bits as:

$$(11010)_2 = h_4 h_3 h_2 h_1 h_0$$

with:

$$h_4 = 1, \qquad h_3 = 1, \qquad h_2 = 0, \qquad h_1 = 1, \qquad h_0 = 0$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Square-and-Multiply Algorithm (cont.)

Step	Operation	Action
#0	x	Initial setting, bit processed: $h_4 = 1$
#1a	$(x^1)^2 = x^2$	SQ, bit processed: h_3
#1b	$x^2 \cdot x = x^3$	MUL, since $h_3 = 1$
#2a	$(x^3)^2 = x^6$	SQ, bit processed: h_2
#2b	x^6	No MUL, since $h_2 = 0$
#3a	$(x^6)^2 = x^{12}$	SQ, bit processed: h_1
#3b	$x^{12} \cdot x = x^{13}$	MUL, since $h_1 = 1$
#4a	$(x^{13})^2 = x^{26}$	SQ, bit processed: h_0
#4b	x^{26}	No MUL, since $h_0 = 0$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Square-and-Multiply Algorithm

Square-and-Multiply for Modular Exponentiation
Input:
base element x
exponent $H = \sum_{i=0}^{t} h_i 2^i$ with $h_i \in 0, 1$ and $h_t = 1$
and modulus n
Output: $x^H \mod n$
Initialization: $r = x$
Algorithm:

1 FOR $i = t - 1$ DOWNTO 0
1.1 $r = r^2 \mod n$
IF $h_i = 1$
1.2 $r = r \cdot x \mod n$
2 RETURN (r)

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Decryption with Chinese Remainder Theorem CRT

Why CRT Speeds Up RSA

Direct decryption computes $m \equiv c^d \pmod{n}$ with $n = pq$. CRT replaces this with two smaller computations modulo p and q.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

CRT: Reduction and Exponentiation

Step 1: Reduce

$$c_p \equiv c \pmod{p}$$
$$c_q \equiv c \pmod{q}$$

Step 2: Reduce the Exponent

$$d_p \equiv d \pmod{p-1}$$
$$d_q \equiv d \pmod{q-1}$$

Step 3: Exponentiate

$$m_p \equiv c_p^{d_p} \pmod{p}$$
$$m_q \equiv c_q^{d_q} \pmod{q}$$

Next

Recombine m_p and m_q with the Chinese Remainder Theorem.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

CRT: Recombination

$$m \equiv q\, q_{(p)}^{-1}\, m_p + p\, p_{(q)}^{-1}\, m_q \pmod{n}$$

$$q_{(p)}^{-1} \equiv q^{-1} \pmod{p}$$

$$p_{(q)}^{-1} \equiv p^{-1} \pmod{q}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Exercise – Chinese Remainder Theorem

Bob has the following RSA private information:

$$p = 11, \qquad q = 13, \qquad d = 103$$

The received ciphertext is:

$$c = 48$$

Decrypt the ciphertext using the Chinese Remainder Theorem (CRT).

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Public-Key Cryptography



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Bruce Schneier



Twofish,
Blowfish

Vincent Rijmen
and Joan Daemen



AES

Modern private
key encryption

Rivest, Shamir
& Aldeman



Public-k
encrypt

Ron
Rivest



Hashing

Whitfield
Diffie



Key
interchange

Phil
Zimmerman



PGP
Encryption

Attacks

RSA Security: Attack Surface



Main Attack Classes

- **Mathematical:** factor a weak or undersized modulus n.
- **Implementation:** incorrect padding or protocol design.
- **Side-channel/fault:** exploit timing, power, cache, or induced errors.
- **Key generation:** weak randomness or reused primes.
- **Key exposure:** steal or leak the private key.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Mathematical Attack: Factoring



n, e, c

Attack Logic

1. Factor the public modulus: $n = pq$.
2. Compute $\phi(n) = (p-1)(q-1)$.
3. Recover the private exponent:

$$d \equiv e^{-1} \pmod{\phi(n)}.$$

4. Decrypt with $m \equiv c^d \pmod{n}$.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Mathematical Attack: From Factors to Private Key

From Factors to the Private Key

If an attacker recovers p and q, the rest is straightforward:

$$\phi(n) = (p-1)(q-1), \qquad d \equiv e^{-1} \pmod{\phi(n)}.$$

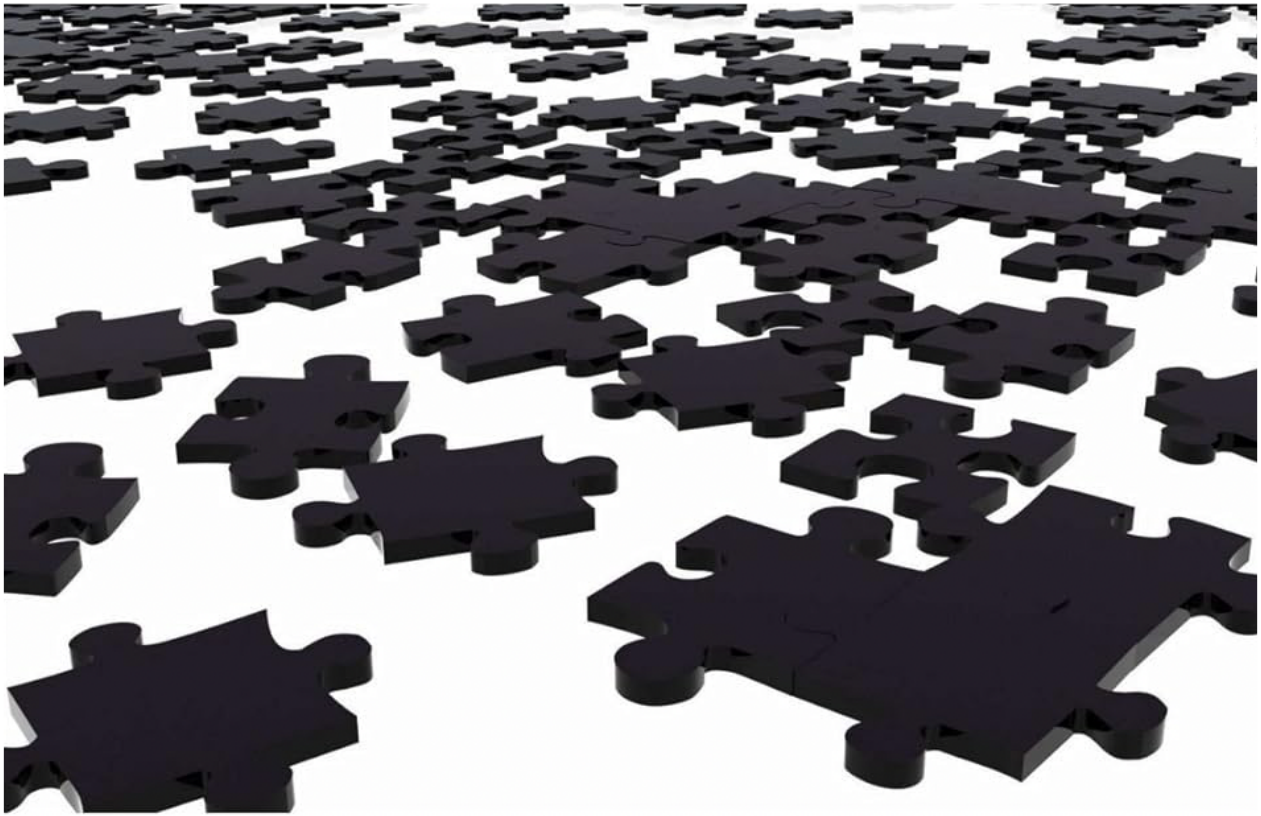
The Extended Euclidean Algorithm computes the modular inverse d.

Security Interpretation

For correctly generated RSA keys, the difficult step is recovering the prime factors from the public modulus $n = pq$.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Factoring Challenges



RSA Factoring Challenge

RSA Security, Computational Number Theory, RSA, RSA Numbers,

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Factoring Challenges



RSA Factoring Challenges

RSA number	Decimal digits	Binary digits	Cash prize offered	Factored on	Factored by
RSA100	100	330	US$1,000[8]	April 1, 1991[9]	Arjen K. Lenstra
RSA110	110	364	US$4,429[8]	April 14, 1992[9]	Arjen K. Lenstra and M.S. Manasse
RSA120	120	397	US$5,898[8]	July 9, 1993[10]	T. Denny *et al.*
RSA129 [a]	129	426	US$100	April 26, 1994[9]	Arjen K. Lenstra *et al.*
RSA130	130	430	US$14,527[8]	April 10, 1996	Arjen K. Lenstra *et al.*
RSA140	140	463	US$17,226	February 2, 1999	Herman te Riele *et al.*
RSA150	150	496		April 16, 2004	Kazumaro Aoki *et al.*
RSA155	155	512	US$9,383[8]	August 22, 1999	Herman te Riele *et al.*
RSA160	160	530		April 1, 2003	Jens Franke *et al.*, University of Bonn
RSA170 [b]	170	563		December 29, 2009	D. Bonenberger and M. Krone [c]
RSA576	174	576	US$10,000	December 3, 2003	Jens Franke *et al.*, University of Bonn
RSA180 [b]	180	596		May 8, 2010	S. A. Danilov and I. A. Popovyan, Moscow State University[11]
RSA190 [b]	190	629		November 8, 2010	A. Timofeev and I. A. Popovyan
RSA640	193	640	US$20,000	November 2, 2005	Jens Franke *et al.*, University of Bonn
RSA200 [b] ?	200	663		May 9, 2005	Jens Franke *et al.*, University of Bonn
RSA210 [b]	210	696		September 26, 2013[12]	Ryan Propper
RSA704 [b]	212	704	US$30,000	July 2, 2012	Shi Bai, Emmanuel Thomé and Paul Zimmermann

MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Factoring Challenges

RSA704 [b]	212	704	US$30,000	July 2, 2012	Shi Bai, Emmanuel Thomé and Paul Zimmermann
RSA220 [b]	220	729		May 13, 2016	S. Bai, P. Gaudry, A. Kruppa, E. Thomé and P. Zimmermann
RSA230 [b]	230	762		August 15, 2018	Samuel S. Gross, Noblis, Inc.
RSA232 [b]	232	768		February 17, 2020[13]	N. L. Zamarashkin, D. A. Zheltkov and S. A. Matveev.
RSA768 [b]	232	768	US$50,000	December 12, 2009	Thorsten Kleinjung *et al.*[14]
RSA240 [b]	240	795		Dec 2, 2019[15]	F. Boudot, P. Gaudry, A. Guillevic, N. Heninger, E. Thomé and P. Zimmermann
RSA250 [b]	250	829		Feb 28, 2020[16]	F. Boudot, P. Gaudry, A. Guillevic, N. Heninger, E. Thomé and P. Zimmermann
RSA260	260	862			
RSA270	270	895			
RSA896	270	896	US$75,000[d]		
RSA280	280	928			
RSA290	290	962			
RSA300	300	995			
RSA309	309	1024			
RSA1024	309	1024	US$100,000[d]		

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Factoring Challenges

RSA310	310	1028		
RSA320	320	1061		
RSA330	330	1094		
RSA340	340	1128		
RSA350	350	1161		
RSA360	360	1194		
RSA370	370	1227		
RSA380	380	1261		
RSA390	390	1294		
RSA400	400	1327		
RSA410	410	1360		
RSA420	420	1393		
RSA430	430	1427		
RSA440	440	1460		
RSA450	450	1493		
RSA460	460	1526		
RSA1536	463	1536	US$150,000[d]	

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

RSA Factoring Challenges

RSA470	470	1559		
RSA480	480	1593		
RSA490	490	1626		
RSA500	500	1659		
RSA617	617	2048		
RSA2048	617	2048	US$200,000[d]	

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Integer Factorization Algorithms (cont.)

Given a composite number

$$n = pq,$$

the goal is to recover the prime factors p and q.

Algorithm	**Main Idea**
Trial Division	Test possible divisors up to $\sqrt{n}$.
Fermat	Write n as a difference of two squares: $n = a^2 - b^2 = (a-b)(a+b)$
Pollard's Rho	Generate a sequence and use $\gcd(\lvert x - y \rvert, n)$ to discover a non-trivial factor.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Factorization Algorithms and RSA (cont.)

Algorithm	**Implementation**	**Best Use**	**Large n**
Trial Division	Very easy	Small numbers	No
Fermat	Easy	p and q are close	No
Pollard's Rho	Medium	Small factors	Limited
Quadratic Sieve	Difficult	Large integers	Yes
GNFS	Very difficult	Very large integers	Yes

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Factorization Algorithms and RSA (cont.)

Connection with RSA
If an attacker can factor:

$$n = pq$$

then the attacker can compute:

$$\phi(n) = (p-1)(q-1)$$

and recover the private exponent:

$$d = e^{-1} \pmod{\phi(n)}$$

$$\boxed{n \longrightarrow p, q \longrightarrow \phi(n) \longrightarrow d}$$

Therefore, RSA requires a modulus that is difficult to factor.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Lab 3: RSA Factorization Attack (cont.)

Recover the RSA private key by factoring the public modulus.

$$n = pq$$

- Implement three factorization algorithms:
  - **Trial Division**
  - **Fermat Factorization**
  - **Pollard's Rho**
- Recover the prime factors p and q.
- Compute:

$$\phi(n) = (p-1)(q-1) \qquad d = e^{-1} \pmod{\phi(n)}$$

- Use the recovered private key (n, d) to decrypt the ciphertext.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Lab 3: RSA Factorization Attack (cont.)

- Compare the execution time of the three algorithms for RSA moduli of different sizes.

Initial test:

$$n = 3233, \qquad e = 17, \qquad c = 2790$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Cryptography

Public-Key Cryptography – RSA

Cristhian Iza

School of Mathematical and Computational Sciences
Yachay Tech University

September 20, 2026

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES